Programming Assignment-2 Report

CS683

Ram Jyothy 24B0712
Anshul Goyal 24B1077
Tanish Niraj Palsapure 24B3946
Sailesh Kumar Sahoo 24B1208

Contents

1 Baseline Assumptions **2**

2 Task 1: Stride Pattern Prefetching (SPP) **2**
2.1 Task 1A: IP-Stride Prefetcher . . . 2
2.1.1 Design Overview . . . 2
2.1.2 Results . . . 3
2.1.3 Flaws . . . 3
2.2 Task 1B: Stride Pattern Prefetcher . . . 3
2.2.1 Design Overview . . . 3
2.2.2 Results . . . 4
2.2.3 Total Storage Overhead . . . 6
2.2.4 Threshold Decisions for Accuracy Throttling . . . 6

3 Task 2: Instruction-Conscious L2C Replacement **7**
3.1 Task 2a: Instruction-Conscious RRIP . . . 7
3.1.1 Motivation and policy design . . . 7
3.1.2 Measurement scope . . . 8
3.1.3 Static-policy results . . . 9
3.1.4 Instruction–data trade-off and final static design . . . 10
3.2 Task 2b: Dynamic I-RRIP . . . 10
3.2.1 Leader sampling and exact decision rule . . . 10
3.2.2 Dynamic-policy results . . . 11
3.2.3 Threshold sensitivity and interpretation . . . 13

4 Task 3: Incorporating SPP and I-RRIP **14**
4.1 Motivation and objective . . . 14
4.2 Replacement treatment of prefetch packets and design rationale . . . 14
4.2.1 Behavior of the evaluated combined policy . . . 14
4.2.2 Prefetch-specific extension . . . 14
4.3 Experimental methodology . . . 15
4.4 Results and analysis . . . 17
4.4.1 IPC against both baselines . . . 17
4.4.2 Prefetch usefulness and miss reduction . . . 18
4.4.3 Pollution and the instruction/data trade-off . . . 18
4.5 Conclusion . . . 18

1 Baseline Assumptions

Table 1: Baseline Configuration

	Property	Value
1	L1I capacity / sets / ways	32 KB / 64 / 8
2	L1I MSHR entries	8
3	L1D capacity / sets / ways	48 KB / 64 / 12
4	L1D MSHR entries	16
5	L2C capacity / sets / ways	512 KB / 1024 / 8
6	L2C MSHR entries	32
7	LLC capacity / sets / ways	2 MB / 2048 / 16
8	LLC MSHR entries	64
9	Cache line size	64 B (all levels)
10	Replacement policy	LRU (all levels)
11	Page size	4 KB

The ChampSim simulation is configured to a single-core processor for all following results. No prefetchers feature at the L2C, LLC or MMU.

Table 2: Baseline Trace Stats

Trace	IPC	L1D Load MPKI	L1D Load Miss Rate (%)
Trace1	0.232	10.68	6.24
Trace2	0.250	13.54	7.84
Trace3	0.382	3.40	1.67

Not much to gain in Trace3. Trace2 seems to be the most promising candidate from this preliminary analysis.

2 Task 1: Stride Pattern Prefetching (SPP)

2.1 Task 1A: IP-Stride Prefetcher

2.1.1 Design Overview

The IP-Stride prefetcher, as implemented in `prefetcher/ip_stride.l1d_pref`, is placed at the L1D cache, and was designed to exploit constant cache-aligned stride between consecutive memory load accesses at the same IP. Cache-aligned array accesses in loops exhibit the same (i.e., A+2s, A+4s, ..., where 's' corresponds to the block size).

It makes use of a 24-entry Prefetch Table indexed by the lower 8 bits of the IP with LRU replacement. Each entry comprises the masked IP, last cache line address, last stride and LRU bit (and possibly information pertaining to OS pages). Prefetches are triggered when the current stride at a given IP matches the last. This requires at least three demand requests from the same IP. Strides that cross the page boundary are discarded (which provides an upper bound of 63 on its magnitude). Zero strides are also discarded.

2.1.2 Results

Table 3: Degree 3 IP Stride Prefetcher Stats

Trace	Config	IPC	Speedup	Load MPKI	Load Miss (%)	Accuracy (%)
Trace1	Baseline	0.2322	1.0000	10.68	6.24	–
	IP stride	0.2333	1.0050	10.12	5.90	52.90
Trace2	Baseline	0.2500	1.0000	13.54	7.84	–
	IP stride	0.2502	1.0006	13.44	7.79	37.81
Trace3	Baseline	0.3819	1.0000	3.40	1.67	–
	IP stride	0.3828	1.0024	3.32	1.63	16.08

2.1.3 Flaws

While providing 100% coverage for constant-stride access streams, the IP-Stride Prefetcher presents a number of flaws. Some of them are listed below:

1. **Lack of Hysteresis:** Prefetch decisions are made based on the previous state only, which is overwritten upon a mismatch. This lack of tolerance/confidence measures causes the prefetcher to provide 0 coverage for access streams that deviate even slightly from the uniform stride case (ex: +1, +2, +1, +2, ... incurs zero prefetches). A feedback mechanism cannot be designed to train the prefetcher to recognize simple non-uniform patterns.

2. **IP-Table Thrashing:** A 24 entry IP-table with LRU replacement is susceptible to thrashing. Indexing into the table using a masked IP also presents a possible aliasing problem.

3. **Fixed Degree:** There is no mechanism whereby prefetcher degree is not throttled by accuracy, which might lead to cache pollution.

4. **Irregular Access Patterns:** Stride patterns like $(+1, +2, +3)^n$ are not adequately handled by the implementation.

   Our implementation of SPP mitigates these.

2.2 Task 1B: Stride Pattern Prefetcher

2.2.1 Design Overview

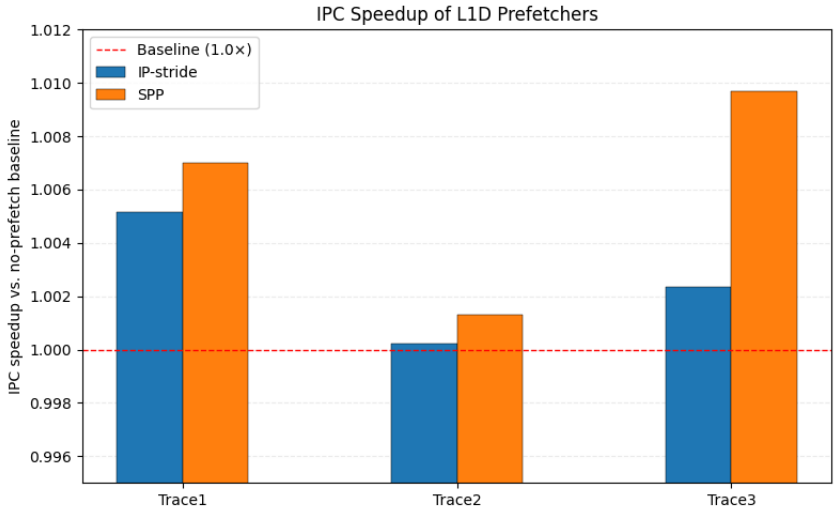
The prefetcher operates in two modes: **uniform and non-uniform**. An IP stride tracker uses a 3-bit saturating confidence counter to dynamically switch between these modes based on whether the predicted stride matches the observed stride. In uniform mode, the prefetcher uses a fixed stride, initialized when entering this mode. In non-uniform mode, it maintains a history of recently observed strides, hashes this history into a Signature Path Prediction (SPP) table, and uses the predicted strides from reliable table entries to generate prefetches. Each SPP table entry maintains its own confidence counter to determine the reliability of its prediction. Additionally, a global accuracy tracker

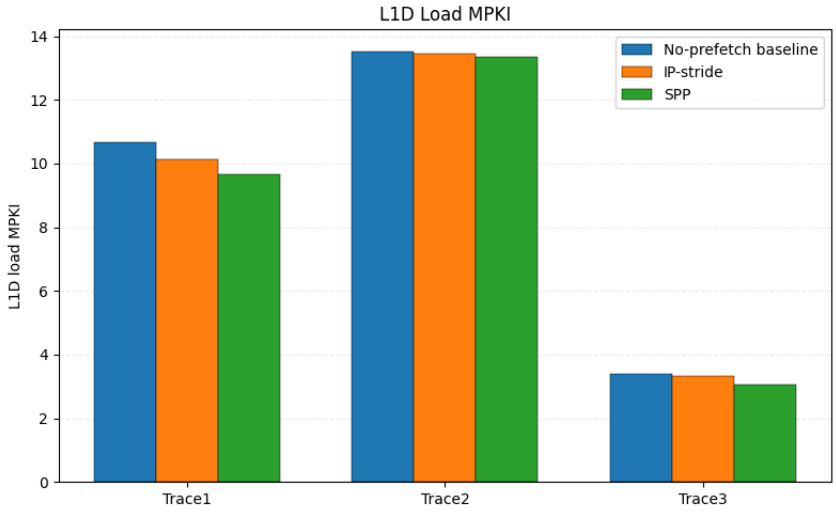
measures prefetch accuracy over epochs of approximately 1000 cycles and dynamically adjusts the prefetch degree between 1 and 5 based on the observed accuracy.

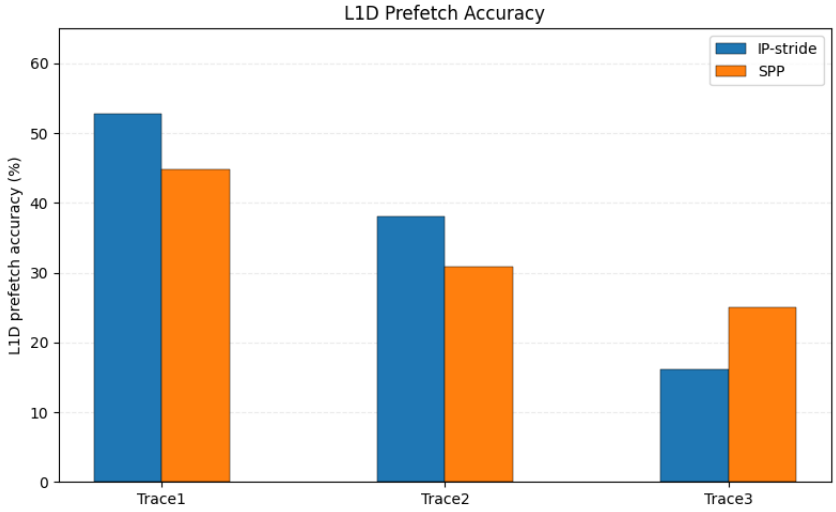
2.2.2 Results

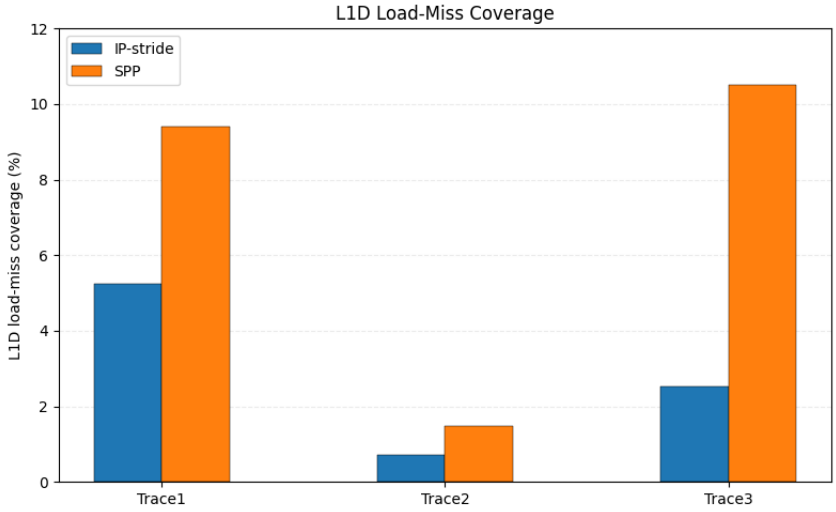
Table 4: Baseline, IP-stride, and SPP comparison across three traces.

Trace	Configuration	IPC	Speedup	L1D load MPKI	ChampSim accuracy	Load-miss coverage
Trace 1	Baseline	0.232204	1.000000×	10.6787	–	0.0000%
Trace 1	IP-stride	0.233398	1.005142×	10.1188	52.7914%	5.2433%
Trace 1	SPP	0.233834	1.007020×	9.67444	44.8282%	9.4045%
Trace 2	Baseline	0.251130	1.000000×	13.5338	–	0.0000%
Trace 2	IP-stride	0.251186	1.000223×	13.4356	38.1276%	0.7253%
Trace 2	SPP	0.251461	1.001318×	13.3348	30.8288%	1.4701%
Trace 3	Baseline	0.382061	1.000000×	3.40324	–	0.0000%
Trace 3	IP-stride	0.382962	1.002358×	3.31768	16.0936%	2.5141%
Trace 3	SPP	0.385764	1.009692×	3.04544	25.0461%	10.5135%
Geometric mean	Baseline	0.281385	1.000000×	7.89363	–	0.0000%
Geometric mean	IP-stride	0.282108	1.002572×	7.66903	31.8776%	2.1224%
Geometric mean	SPP	0.283074	1.006004×	7.32410	32.5899%	5.2579%




L1D Load MPKI
No-prefetch baseline
IP-stride
SPP
L1D load MPKI
0
2
4
6
8
10
12
14
Trace1
Trace2
Trace3


L1D Prefetch Accuracy
IP-stride
SPP
L1D prefetch accuracy (%)
0
10
20
30
40
50
60
Trace1
Trace2
Trace3


L1D Load-Miss Coverage
IP-stride
SPP
L1D load-miss coverage (%)
0
2
4
6
8
10
12
Trace1
Trace2
Trace3

2.2.3 Total Storage Overhead

Each `IPStrideTracker` requires 69 bits of storage, comprising 16 bits for the instruction pointer (IP), 16 bits for the last cache line, 12 bits for the history signature, 5 bits for the LRU state, 2 bits for the timer, 7 bits for the uniform stride, and 11 bits for the switcher.

Each entry in the SPP table requires 10 bits, consisting of a 3-bit confidence counter and a 7-bit stride field.

The epoch timers and accuracy-tracking mechanism require an additional 45 bits: 10 bits for the epoch timer, 16 bits each for the epoch useful and useless counts, and 3 bits for the current prefetch degree.

With 24 `IPStrideTracker` entries and 32 SPP table entries, the total storage overhead is

$$24 \times 69 + 32 \times 10 + 45 = 2021 \text{ bits.}$$

This corresponds to approximately 253 bytes of storage, accounting for around 0.5147% of the L1D cache capacity.

2.2.4 Threshold Decisions for Accuracy Throttling

During our experiments, we observed that the accuracy mostly remained below 50%. This motivated us to explore accuracy thresholds within this range for throttling prefetching. We evaluated multiple combinations of thresholds and prefetch degrees across different workloads.

The best overall results were obtained with the following configuration:

Prefetch degree mapping: 1/2/3/4/5
Accuracy thresholds: 10%/20%/30%/40%

This configuration adapts the prefetch degree to observed accuracy while retaining relatively aggressive prefetching behaviour.

Tables 5 and 6 summarize the configurations evaluated and their corresponding IPC results.

Table 5: SPP accuracy-threshold and prefetch-degree configurations evaluated.

Degree mapping	Accuracy thresholds (%)
1/2/3/4/5	20/40/60/80, 10/20/30/40, 10/20/40/70, 30/60/80/90
1/1/2/2/3	20/40/60/80, 10/20/30/40, 10/30/50/70, 30/60/80/90
1/1/3/5/5	20/40/60/80, 10/20/30/40
1/1/1/2/2	20/40/60/80

Table 6: IPC for the evaluated SPP configurations across three traces.

Thresholds (%)	Degree mapping	Trace 1	Trace 2	Trace 3
–	No prefetcher	0.232204	0.251130	0.382061
–	IP-stride	0.233398	0.251186	0.382962
Degree mapping: 1/2/3/4/5				
20/40/60/80		0.233886	0.251479	0.385649
10/20/30/40		0.233834	0.251461	0.385764
Degree mapping: 1/1/2/2/3				
20/40/60/80		0.234024	0.251401	0.384963
10/20/30/40		0.234043	0.251392	0.385109
10/30/50/70		0.234013	–	–
30/60/80/90		0.234027	–	–
Degree mapping: 1/1/3/5/5				
20/40/60/80		0.233894	0.251487	0.385594
10/20/30/40		0.233861	0.251475	0.385697
Degree mapping: 1/1/1/2/2				
20/40/60/80		0.234024	–	–

3 Task 2: Instruction-Conscious L2C Replacement

3.1 Task 2a: Instruction-Conscious RRIP

3.1.1 Motivation and policy design

Instruction misses can interrupt the supply of work to the processor. I-RRIP favors instructions in L2 insertion, hit promotion and eviction, trading fewer instruction misses for potentially more data misses.

The replacement-update interface gains a trailing `uint8_t isInstruction`. All eight cache callback sites pass the corresponding `packet.instruction` flag, including hits and MSHR fills; the three BTB callbacks pass zero. Declarations and policy definitions agree, while LRU and ordinary RRIP ignore the new flag. Insertion/promotion use the incoming packet flag. Eviction uses the resident block's instruction bit, set at fill time; these are distinct classifications, especially for translation traffic. No prefetcher is enabled in Task 2.

Two-bit RRIP assigns each line a re-reference prediction value (RRPV) from 0 (protected) to 3 (eligible for eviction). Ordinary SRRIP inserts at 2 and promotes hits to 0. If no line is at 3, it increments all values until a victim exists; ties follow way order. The following variants are cumulative.

V1: instruction insertion. Instruction-flagged fills enter at 0; other fills remain at 2. Instructions survive more ageing rounds before reuse, helping code displaced from L1I. The cost is protection before reuse is demonstrated: one-time code can displace useful data. Hit promotion and eviction remain SRRIP.

V2: asymmetric promotion. Instruction-flagged hits reset RRPV to 0, while other hits decrement it by one, saturating at 0. Relative to V1, the change is weaker *data* pro-

motion; instruction hits already received full promotion. Repeated data reuse still earns protection, but a single data hit does not necessarily compete equally with instruction reuse. This can penalize useful data with sparse accesses.

V3: instruction-aware eviction. Among resident lines at RRPV 3, select a non-instruction line first, otherwise the first line at 3. This preserves instructions only when their RRIP priority ties with data. It neither protects instructions indefinitely nor ages a lower-RRPV data line merely to avoid evicting an instruction at 3. The cost is losing some equally ranked data reuse. The audited policies select invalid ways first. RRPVs need two logical bits per line, although the static C++ arrays use 32-bit entries; the existing instruction bit is reused.

3.1.2 Measurement scope

Each trace uses 25M warmup and 25M measured instructions, one core, a 512 KiB 8-way L2, LRU elsewhere and no prefetching. In the measured region, speedup is IPC divided by same-run LRU IPC, and load MPKI is $1000\times$ load misses / retired instructions. Instruction and data load MPKI partition L2C load MPKI. Task 2a retains its historical 15-run dataset; Task 2b uses a separate matched full run. No cross-run speedups are computed.

3.1.3 Static-policy results

Table 7: Task 2a: historical static-policy run, with its own LRU baseline. MPKI columns count L2 loads; component rounding can differ from the total by 0.001.

Trace	Policy	IPC	Speedup ×LRU	L2C Load MPKI	Instr. MPKI	Data MPKI
Trace1	LRU	0.232345	1.0000	17.696	10.338	7.358
	RRIP	0.232231	0.9995	17.639	10.562	7.077
	I-RRIP V1	0.232256	0.9996	17.682	10.115	7.567
	I-RRIP V2	0.232223	0.9995	17.695	9.973	7.722
	I-RRIP V3	0.232109	0.9990	17.753	9.850	7.902
Trace2	LRU	0.251158	1.0000	27.357	18.853	8.504
	RRIP	0.251806	1.0026	25.746	17.781	7.965
	I-RRIP V1	0.253964	1.0112	24.848	16.116	8.732
	I-RRIP V2	0.255011	1.0153	23.894	14.750	9.144
	I-RRIP V3	0.256535	1.0214	22.602	13.259	9.344
Trace3	LRU	0.382065	1.0000	3.954	1.124	2.830
	RRIP	0.381739	0.9991	4.039	1.197	2.842
	I-RRIP V1	0.382002	0.9998	3.948	1.077	2.871
	I-RRIP V2	0.382079	1.0000	3.896	1.024	2.871
	I-RRIP V3	0.382079	1.0000	3.878	0.995	2.883

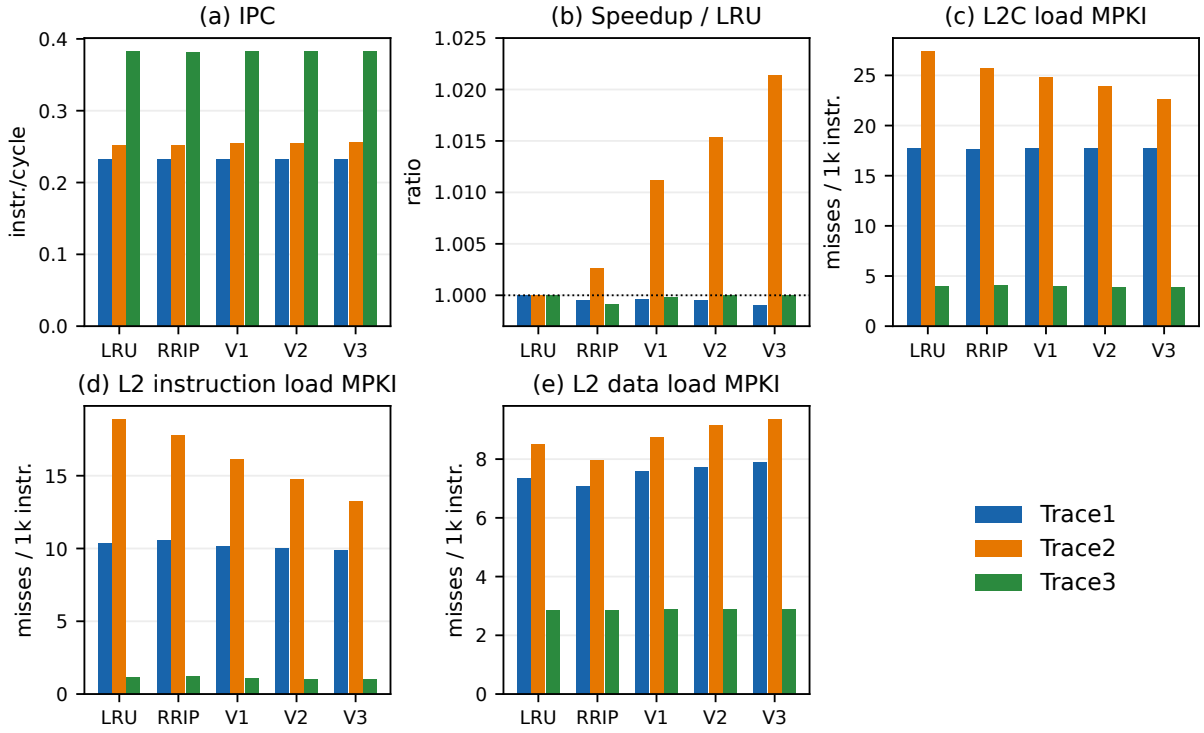


Figure 1: Task 2a: all five required metrics from the historical logs used in Table 7. V1–V3 denote the cumulative variants. The speedup panel uses a truncated vertical scale to show small changes; the IPC and MPKI bars start at zero.

3.1.4 Instruction–data trade-off and final static design

Trace2 benefits most: V3 reduces instruction MPKI from 18.853 to 13.258 (−29.7%), while data MPKI rises from 8.504 to 9.344 (+9.9%). Total load MPKI falls by 17.4% and IPC improves by 2.14% over LRU, or 1.88% over RRIP. V1, V2 and V3 progressively improve IPC. Compare RRIP to V1 to assess insertion; comparing LRU to V1 also changes the baseline policy. These cumulative comparisons do not isolate interactions among modifications.

On Trace1, instruction MPKI falls by 4.7%, but data MPKI rises by 7.4%; total MPKI increases slightly and IPC falls by 0.10%. Trace3 begins with only 1.124 instruction MPKI. V3 reduces it to 0.995, with data MPKI rising from 2.830 to 2.883; IPC changes by less than 0.01%. Thus, fewer instruction misses do not guarantee higher IPC. V3 is the final static design because it gives the lowest instruction MPKI on all three traces and the highest aggregate speedup of the tested variants, driven by Trace2; it is not uniformly beneficial.

3.2 Task 2b: Dynamic I-RRIP

3.2.1 Leader sampling and exact decision rule

A dynamic selector attempts to retain instruction preference only when useful. For set index s, the implementation chooses leaders using $s \bmod 32 = 16$: sets $16, 48, 80, \ldots, 1008$. These 32 sets (3.125% of L2) always use RRIP; the other 992 are followers. Deterministic spacing distributes samples without a leader table, and the small fraction limits fixed-RRIP capacity. This is a design rationale, not a demonstrated optimum: no leader-count sweep was performed. There is one RRIP leader group, not two competing policy groups as in conventional set dueling.

Four per-core cumulative counters record instruction-flagged accesses/misses (I_A, I_M) and non-instruction accesses/misses (D_A, D_M) at leaders. Each successful hit callback increments one access; each completed fill increments one access and one miss. Victim searches and failed retries do not sample; MSHR merges contribute their eventual fill. All callback request types count, so these counters are not the load-only MPKI counters. A miss intentionally receives twice a hit's weight in the exact implemented metric:

$$q = \frac{I_A + I_M}{I_A + D_A + I_M + D_M}, \qquad \text{follower policy} = \begin{cases} \text{I-RRIP V3}, & q \geq \theta, \\ \text{RRIP}, & q < \theta. \end{cases} \tag{1}$$

With no samples, $q = 0$ and the initial mode is RRIP. After each leader update, the mode is recomputed, including equality in the V3 case. The current leader still uses RRIP; subsequent follower insertion, promotion and eviction use the selected mode. Switching preserves RRPVs, avoiding a cache flush or scan. Invalid ways remain first choices. Counters span warmup and simulation without reset or decay; there is no hysteresis. This reduces control work but can slow phase adaptation. Only leader updates recompute the ratio. The configurable threshold defaults to 0.25 and accepts finite values in $[0, 1]$.

3.2.2 Dynamic-policy results

Table 8: Task 2b: matched full-run controls and all five dynamic thresholds. Speedup uses this run's LRU, not Task 2a's baseline. MPKI columns count L2 loads.

Trace	Policy	IPC	Speedup ×LRU	L2C Load MPKI	Instr. Load MPKI	Data Load MPKI
Trace1	LRU	0.232288	1.0000	17.751	10.403	7.348
	RRIP	0.232320	1.0001	17.681	10.613	7.068
	I-RRIP V3	0.232193	0.9996	17.772	9.869	7.902
	Dyn. 0.10	0.231939	0.9985	17.785	9.906	7.879
	Dyn. 0.20	0.232118	0.9993	17.769	9.893	7.876
	Dyn. 0.25	0.231952	0.9986	17.774	9.900	7.874
	Dyn. 0.35	0.231955	0.9986	17.778	9.901	7.877
	Dyn. 0.50	0.232319	1.0001	17.682	10.613	7.069
Trace2	LRU	0.250638	1.0000	28.229	19.530	8.699
	RRIP	0.251296	1.0026	26.607	18.546	8.061
	I-RRIP V3	0.256319	1.0227	23.236	13.743	9.493
	Dyn. 0.10	0.256134	1.0219	23.353	13.900	9.453
	Dyn. 0.20	0.256133	1.0219	23.353	13.900	9.454
	Dyn. 0.25	0.256134	1.0219	23.353	13.900	9.453
	Dyn. 0.35	0.255727	1.0203	23.688	14.264	9.424
	Dyn. 0.50	0.251382	1.0030	26.603	18.548	8.055
Trace3	LRU	0.382008	1.0000	3.959	1.123	2.836
	RRIP	0.381631	0.9990	4.044	1.198	2.847
	I-RRIP V3	0.382057	1.0001	3.895	1.000	2.895
	Dyn. 0.10	0.381999	1.0000	3.900	1.007	2.893
	Dyn. 0.20	0.382010	1.0000	3.900	1.007	2.894
	Dyn. 0.25	0.382001	1.0000	3.901	1.006	2.894
	Dyn. 0.35	0.381775	0.9994	3.983	1.097	2.886
	Dyn. 0.50	0.381631	0.9990	4.044	1.198	2.847

Run provenance and verification. The Task 2b dataset is the full run dated 15 October 2026, built with GCC 11.4. Its 30 matrix logs, source snapshot and CSV are consistent; the repeated dynamic-0.25 Trace1 run has identical parsed statistics. Task 2a's logs were recovered from commit `395678d` and independently checked. The earlier launch paths select seed token `traces`, while Task 2b's stable aliases select `Trace1`, `Trace2` or `Trace3` in the simulator's filename-derived seeding code. Source comparison also finds the later invalid-way-first correction. These establish configuration differences; their individual contribution to the numerical shifts is unresolved.

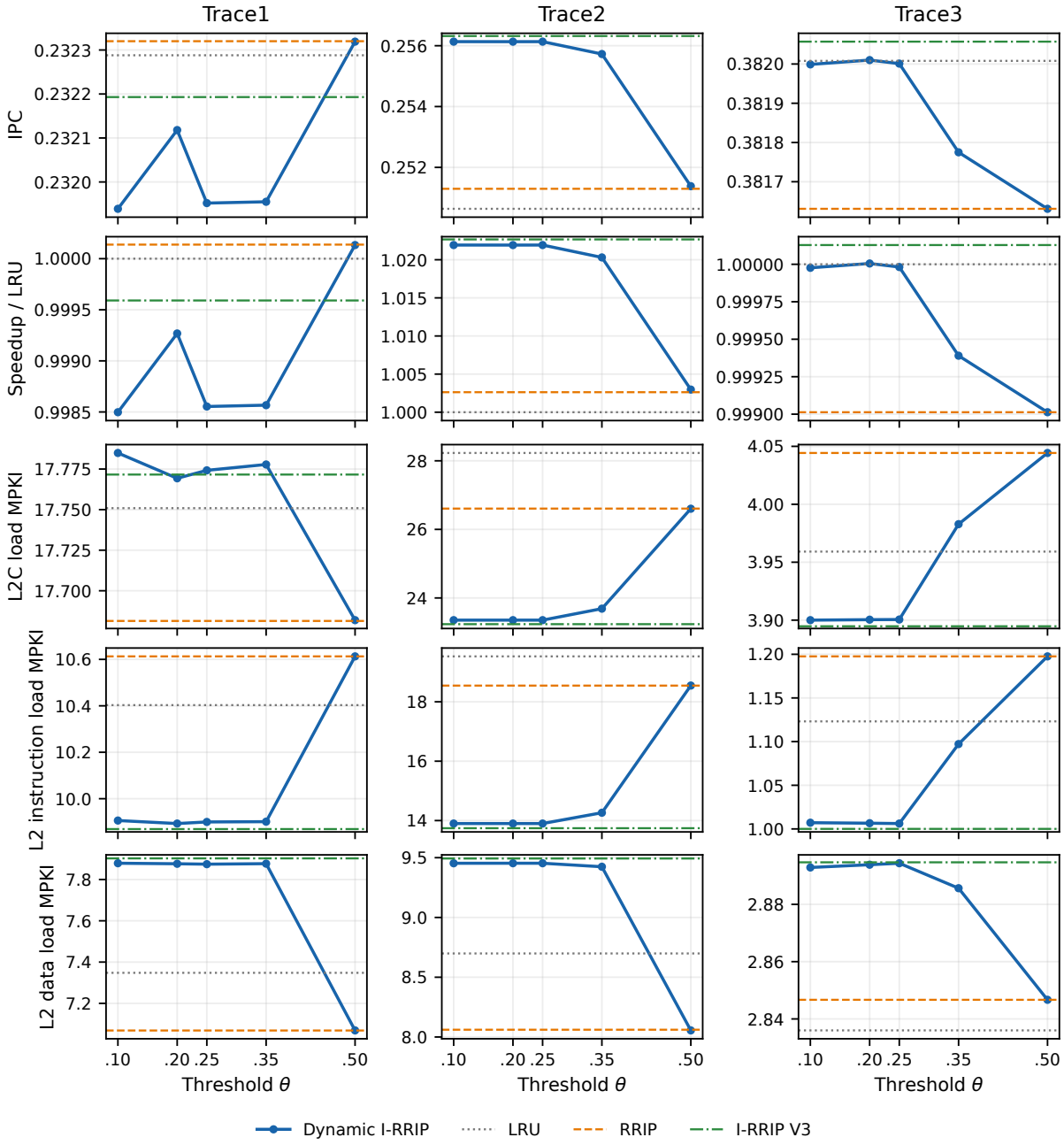


Figure 2: Task 2b threshold sensitivity for all five required metrics and all three traces. Points are the five measured thresholds; connecting segments do not represent additional simulations. Horizontal lines are same-run LRU, RRIP and static V3. IPC is instructions/cycle, speedup is relative to LRU, and every MPKI axis is load misses per 1000 retired instructions. Vertical scales vary between panels and are magnified to expose small differences.

3.2.3 Threshold sensitivity and interpretation

Table 8 and Figure 2 show that the selector mostly interpolates between RRIP-like and V3-like behavior. Final leader metrics vary little *across thresholds for a given trace*: approximately 0.384 for Trace1, 0.480 for Trace2 and 0.319 for Trace3. These endpoint diagnostics do not establish a constant metric throughout execution.

Thresholds 0.10, 0.20 and 0.25. All traces finish in V3 mode, but their histories differ. On Trace1, the fractions of follower updates using V3 are about 99.2%, 96.8% and 95.7%, respectively; threshold 0.20 gives the best IPC of these three despite less V3 use. All reduce instruction MPKI to about 9.9 while increasing data MPKI to about 7.9, and all lose slightly to LRU. On Trace2, over 99.97% of follower updates use V3 for each threshold. Speedup is approximately 1.0219, with instruction/data MPKI near 13.900/9.453, compared with V3's 13.743/9.493. On Trace3, V3-use fractions decrease from about 93.8% to 89.7% to 87.2%; instruction MPKI stays near 1.007 and IPC remains close to LRU. The differing histories and permanently RRIP leaders prevent exact equivalence to static V3.

Threshold 0.35. Trace1 and Trace2 finish in V3, while Trace3 finishes in RRIP. On Trace2, 11 switches and 158,335 RRIP follower updates accompany higher instruction MPKI (14.264) and lower speedup (1.0203) than the lower thresholds; data MPKI falls slightly to 9.424. On Trace3, roughly 43% of follower updates still use V3, despite the final RRIP mode. Its instruction MPKI of 1.097 and speedup of 0.9994 lie between the controls. Final mode alone therefore does not describe the measured interval.

Threshold 0.50. All traces finish in RRIP. Trace1 and Trace3 nearly reproduce RRIP's MPKI and IPC. Trace2 still records 40 switches and 396,257 V3 follower updates; its instruction/data MPKI becomes 18.548/8.055 and speedup falls to 1.0030. Thus a high threshold removes most of Trace2's instruction-retention benefit. Trace1 at 0.50 *does* exceed static V3 (0.232319 versus 0.232193 IPC), but does not exceed ordinary RRIP (0.232320).

Aggregate outcome and limitations. Static V3 achieves geometric-mean speedup **1.0074**; the best tested dynamic threshold, **0.20**, achieves **1.0070** (RRIP: 1.0006). The selector does not improve the aggregate result. Its instruction-weight proxy does not directly measure the IPC benefit of protecting instructions; low thresholds also enable V3 on Trace1, whereas 0.50 sacrifices the large Trace2 gain. Fixed RRIP leaders and cumulative switching histories further distinguish dynamic from static V3. Their individual performance costs were not isolated. No tested threshold selects V3 finally on Trace2 alone: 0.35 separates Trace3 but not Trace1. Intermediate thresholds might separate the final metrics, but are untested and cannot establish an IPC advantage. Three traces, one measured window each and no variance estimate limit generalization; repeatability does not establish statistical significance.

4 Task 3: Incorporating SPP and I-RRIP

4.1 Motivation and objective

SPP fills the hierarchy with lines that no demand request has asked for yet. At the L2, these lines compete for the same eight ways as instruction and data demand fills, and Dynamic I-RRIP (Section 3) already decides how strongly to protect instructions relative to data. A prefetched line is a prediction, not demonstrated reuse. Inserting it like a demand fill lets inaccurate prefetches displace useful lines (pollution), while inserting it too weakly can evict a correct prefetch before its demand arrives (premature eviction). Task 3 asks whether the L2 policy should treat `PREFETCH` packets differently, with the goal of exceeding the IPC of both Dynamic I-RRIP without SPP and standalone SPP.

4.2 Replacement treatment of prefetch packets and design rationale

4.2.1 Behavior of the evaluated combined policy

The measured combined configuration runs the team's SPP at the L1D and the Task 2b Dynamic I-RRIP at the L2 (threshold 0.25, 32 RRIP leader sets). Its replacement code receives the packet `type` in both callbacks but does not branch on it: every decision is keyed on the instruction flag. SPP issues L1D data prefetches, so a `PREFETCH` packet reaching the L2 carries `isInstruction` $= 0$ and is handled exactly like a data request:

- **Insertion.** A prefetch fill enters at RRPV 2 in both RRIP and I-RRIP V3 modes, the same position as a demand data fill. Only instruction fills in V3 mode enter at 0.
- **Promotion.** Any hit, demand or prefetch, resets RRPV to 0 in RRIP mode. In V3 mode a hit on a data line, including a prefetched line, decrements RRPV by one. A redundant prefetch that hits a resident line therefore promotes it like a demand hit.
- **Eviction.** Invalid ways are chosen first. In V3 mode, a non-instruction line at RRPV 3 is chosen before an instruction line at RRPV 3, so prefetched lines are evicted ahead of equally aged instructions. RRIP mode takes the first line at RRPV 3.
- **Mode selection.** In leader sets, prefetch fills and hits count as non-instruction accesses (and misses for fills) in Equation 1. Heavy prefetching therefore lowers q and biases followers towards RRIP.
- **Metadata.** The policy keeps no prefetch-specific state. The cache's resident `BLOCK::prefetch` bit exists but is not read by the policy.

This treatment already gives prefetches lower priority than instructions in V3 mode through the instruction-aware tie-break, but no lower priority than demand data.

4.2.2 Prefetch-specific extension

The team also specified a prefetch-aware variant of the follower-set policy, selected by a compile-time switch and leaving leader sets unchanged:

1. A data line filled by a `PREFETCH` packet is inserted at RRPV 3 instead of 2.
2. A `PREFETCH` hit on a resident line does not promote it.
3. A demand hit on a line still marked as prefetched is promoted to RRPV 0 and the line is thereafter treated as demand data.

4. Leader sampling is unchanged, so prefetch accesses remain non-instruction samples as in Task 2b.

The rationale follows from SPP's measured behavior: at most a few percent of issued prefetches are later used at the L1D (Table 9). Distant insertion makes an unused prefetch the first victim, and combined with the V3 tie-break, prefetches are sacrificed before both instructions and demand data. Suppressing promotion on prefetch hits stops repeated predictions of the same line from manufacturing reuse evidence. Full promotion on the first demand hit compensates for the weak insertion once the prediction is confirmed. The trade-off is premature eviction: a correct but early prefetch inserted at RRPV 3 can be evicted before its demand arrives, which is most likely in Trace2 where L2 pressure is highest.

This extension has not been evaluated. The run labelled "prefetch-aware" in the Task 3 dataset matches SPP + Dynamic I-RRIP in every recorded statistic on all three traces, and the committed `dyn_irrip.l2c_repl` does not contain the switch. No valid measurement of the extension therefore exists, and all results below concern the combined policy described in the previous subsection.

4.3 Experimental methodology

All runs use the single-core configuration of Table 1, 25M warmup and 25M simulated instructions per trace, and Trace1–Trace3. The SPP is the team's Task 1B implementation imported unchanged (commit `ac2ab4c`); it issues prefetches to the L1D, or to the L2 when L1D MSHR occupancy is at least half. No L2C, LLC or TLB prefetcher is enabled, and L1I, L1D and LLC use LRU. Four matched configurations were run on the same build setup:

Configuration	L1D prefetcher	L2 replacement
LRU (reference)	none	LRU
Dyn. I-RRIP	none	Dynamic I-RRIP, $\theta = 0.25$
SPP + LRU (standalone SPP)	SPP	LRU
SPP + Dyn. I-RRIP (combined)	SPP	Dynamic I-RRIP, $\theta = 0.25$

These controls were rerun for Task 3; no Task 2 values are mixed in, so the LRU IPCs differ slightly from Table 8.

Metrics. IPC is the cumulative measured-region IPC. Speedup is IPC divided by the same trace's LRU IPC without prefetching. Load MPKI is 1000× load misses / measured instructions; L2 instruction and data load MPKI sum to L2C load MPKI within 10^{-4} for every row. Prefetch metrics use the L1D `PREFETCH` counters, which ChampSim resets at the end of warmup:

$$\text{Accuracy} = \frac{\text{USEFUL}}{\text{ISSUED}}, \qquad \text{Coverage} = \frac{\text{USEFUL}}{\text{USEFUL} + \text{L1D load misses}}, \tag{2}$$

with L1D load misses = L1D load MPKI × 25,000. `ISSUED` counts every request accepted into the L1D prefetch queue, including requests later found to hit in the L1D, so this accuracy is a conservative lower bound. SPP's own "resolved accuracy", useful/(useful + useless), is 44.5–53.5% but includes warmup and excludes unresolved prefetches, so it is not used in the table. Coverage measures the fraction of potential L1D load misses

removed by useful prefetches. Usefulness of lines filled only into the L2 is not captured by either metric. Prefetch metrics are N/A without SPP.

4.4 Results and analysis

Table 9: Task 3: matched runs, 25M warmup + 25M measured instructions. Speedup is relative to LRU without prefetching. Accuracy and coverage follow Equation 2. MPKI columns count loads.

Trace	Configuration	IPC	Speedup ×LRU	Acc. (%)	Cov. (%)	L1D Load MPKI	L2C Load MPKI	L2 Instr. Load MPKI	L2 Data Load MPKI
Trace1	LRU	0.232113	1.0000	N/A	N/A	10.679	17.725	10.367	7.359
	Dyn. I-RRIP	0.231858	0.9989	N/A	N/A	10.679	17.762	9.879	7.883
	SPP + LRU	0.233427	1.0057	3.92	12.25	9.669	16.876	10.452	6.424
	SPP + Dyn.	**0.233501**	**1.0060**	3.90	12.21	9.672	16.824	9.921	6.902
Trace2	LRU	0.249982	1.0000	N/A	N/A	13.537	28.550	19.844	8.706
	Dyn. I-RRIP	0.255322	1.0214	N/A	N/A	13.542	23.824	14.385	9.439
	SPP + LRU	0.250424	1.0018	2.12	2.95	13.340	28.517	20.031	8.487
	SPP + Dyn.	**0.256101**	**1.0245**	2.12	2.96	13.347	23.510	14.256	9.254
Trace3	LRU	0.381868	1.0000	N/A	N/A	3.404	3.961	1.127	2.835
	Dyn. I-RRIP	0.381806	0.9998	N/A	N/A	3.403	3.889	1.002	2.887
	SPP + LRU	0.385487	1.0095	1.18	11.91	3.046	3.436	1.159	2.277
	SPP + Dyn.	**0.385577**	**1.0097**	1.17	11.81	3.050	3.344	1.023	2.321
Geomean	Dyn. I-RRIP		1.0066						
	SPP + LRU		1.0056						
	SPP + Dyn.		**1.0134**						

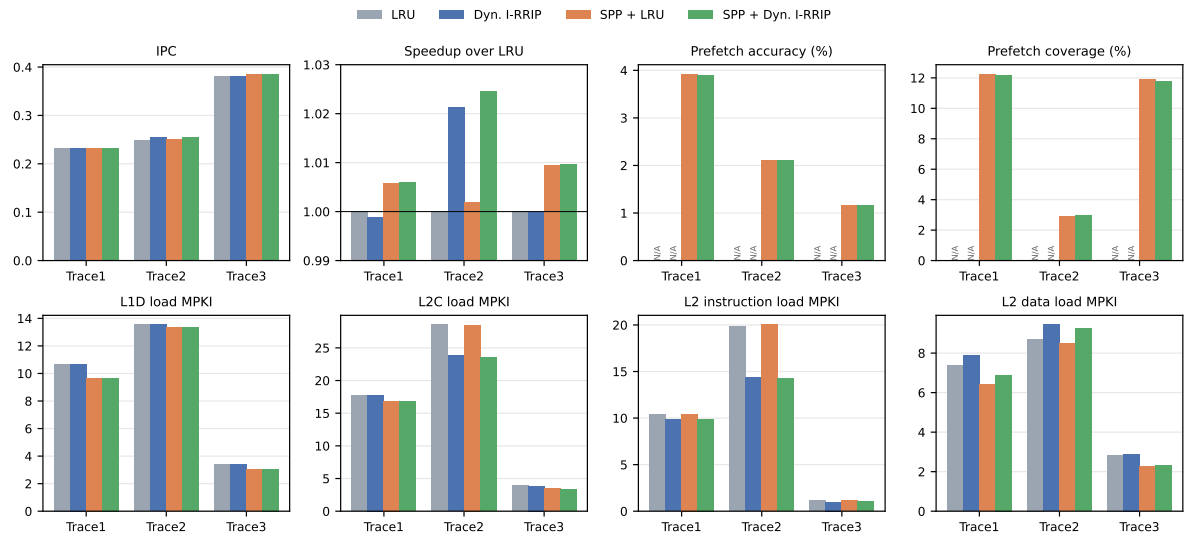


Figure 3: Task 3: all eight required metrics for the four matched configurations of Table 9. Speedup uses a truncated vertical scale with the LRU reference at 1.0; all other axes start at zero. Prefetch accuracy and coverage are N/A without SPP.

4.4.1 IPC against both baselines

Relative to Dynamic I-RRIP without SPP, the combined policy raises IPC by 0.71%, 0.31% and 0.99% on Trace1, Trace2 and Trace3. Relative to standalone SPP, it raises

IPC by 0.03%, 2.27% and 0.02%. The geometric-mean speedup rises from 1.0066 (Dynamic I-RRIP) and 1.0056 (SPP + LRU) to 1.0134. The combined configuration is therefore numerically best on every trace, but the margins over SPP on Trace1 and Trace3 are about 0.03% and fall within the run-to-run sensitivity expected from a single 25M-instruction window. Only Trace2 shows a clear gain over SPP, and that gain comes from the replacement policy rather than from prefetch handling.

The two effects are approximately additive. Dynamic I-RRIP alone contributes +2.14% on Trace2 and slightly reduces IPC elsewhere (−0.11% and −0.02%); SPP alone contributes +0.57%, +0.18% and +0.95%. The combined speedups (1.0060, 1.0245, 1.0097) are close to, and slightly above, the sum of these individual gains.

4.4.2 Prefetch usefulness and miss reduction

SPP reduces L1D load MPKI by 9.5%, 1.5% and 10.5% under LRU, and by almost exactly the same amounts under Dynamic I-RRIP. Accuracy and coverage differ by at most 0.1 percentage points between the two L2 policies. The L2 policy therefore does not change what SPP achieves at the L1D, which is expected because the evaluated policy never distinguishes prefetches. IPC gain from SPP tracks coverage: Trace2, with 2.95% coverage, gains only 0.18%, while Trace1 and Trace3, with about 12% coverage, gain 0.57% and 0.95%. Trace3 has the lowest accuracy yet the largest gain, so coverage, not accuracy, best predicts benefit in this dataset.

4.4.3 Pollution and the instruction/data trade-off

The useful/issued accuracy of 1.2–3.9% means hundreds of thousands of prefetches per run are never consumed at the L1D. Evidence of L2 pollution is visible under LRU: adding SPP lowers L2 data MPKI (by 12.7%, 2.5% and 19.7%) but raises L2 instruction MPKI by 0.8%, 0.9% and 2.9%. Under Dynamic I-RRIP, the V3 tie-break evicts data, including prefetched lines, before instructions, and the combined policy's L2 instruction MPKI is 5.1%, 28.8% and 11.7% lower than standalone SPP's. The cost is higher L2 data MPKI (by 7.4%, 9.0% and 1.9%), but total L2C load MPKI is still lower on all three traces (by 0.3%, 17.6% and 2.7%). The combined policy thus inherits Task 2's trade-off: instruction misses fall and data misses rise, and IPC improves where instruction misses dominate (Trace2).

Because prefetch fills enter at the same RRPV as demand data, the evaluated policy cannot separate useless prefetches from demand data in either mode. Given the low accuracy, a prefetch-aware insertion such as the one specified above is a plausible way to reduce the remaining data-side pressure, but this is an interpretation; it was not measured.

4.5 Conclusion

Combining SPP with Dynamic I-RRIP gave the highest IPC on all three traces (geometric-mean speedup 1.0134 versus 1.0066 and 1.0056), so the stated objective is met numerically by the combination. The clear improvement is on Trace2 (+2.27% over SPP, +0.31% over Dynamic I-RRIP); the gains over SPP on Trace1 and Trace3 are too small to distinguish from noise, and no workload regressed. This gain arises from instruction-conscious L2 replacement protecting instructions from SPP's low-accuracy prefetches, not from any prefetch-specific rule: the evaluated policy treats `PREFETCH` packets as ordinary data.

The prefetch-aware extension was specified but not validly measured, so no measurable advantage of prefetch-aware replacement can be claimed. The main limitations are this missing evaluation, three traces with a single measured window each, prefetch metrics restricted to the L1D, and an accuracy metric that is a lower bound.